

LECTURE 3

Operating System Services and Interfaces

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

- The services every OS provides, grouped by who benefits: the user, or the system itself
- User interfaces — CLI, GUI, and mobile — and why experienced users still reach for the shell
- What a system call is, and how control moves from a user program into the kernel and back
- The three general methods an OS uses to pass parameters into a system call
- The major categories of system calls, illustrated with `fork()`, `exec()`, `open()`, `read()`

OS Services: Convenient for the User

Service	What it means
Program execution	Load a program into memory, run it, and end its execution — normally, or abnormally on error
I/O operations	Users can't control I/O devices directly (efficiency and protection), so the OS performs I/O on their behalf
File-system manipulation	Read, write, create, delete, search, and list files and directories, and manage permissions
Communications	Exchange information between processes, locally or across a network, via shared memory or message passing
Error detection	Watch for CPU/memory errors, I/O failures, and user-program errors, and take correct, consistent action

OS Services: Keeping the System Itself Efficient

Rarely visible to the user directly, but essential when many users or programs share one machine:

Resource allocation

Allocate CPU cycles, memory, storage, and I/O devices among concurrently running jobs

Accounting

Track which users and programs use which resources, and how much, for monitoring or billing

Protection & security

Protection controls access within a trusted system; security defends against outsiders

Protection vs. security — a distinction worth keeping precise

Protection is about controlling access within a trusted system — stopping one legitimate process from reading another's memory without permission. Security defends the system from the outside — authentication, defending against intrusion. A system can have excellent protection and still be breached through weak security, or vice versa.

User Interfaces: The Command-Line Interface (CLI)

- **A CLI, or shell, —** lets a user type commands directly as text, which are then executed
- **Two execution paths —** code built directly into the shell itself, or (far more common on UNIX-family systems) a separate program that implements the command
- **bash and zsh —** almost every command they run (ls, grep, cat) is a genuinely separate executable, not shell-internal code
- **The shell's real job —** mostly parsing what you typed and launching the right program with the right arguments

Graphical and Mobile Interfaces

- **GUI** — replaces typed commands with a mouse-and-window-driven system: icons, windows, and menus a user points at, clicks, and drags
- **Popularized by the early Macintosh** — building on research originally done at Xerox PARC; remains the default interface for most desktop/laptop OSes
- **Mobile / touch** — taps, swipes, and pinches on a screen that is also the entire display, with no separate pointing device
- **Genuinely distinct, not just a small GUI** — targets must be large enough to tap accurately, and screen space is far more limited

Three Interface Styles

CLI

Typed commands; precise, scriptable, composable

GUI

Mouse-driven windows, icons, and menus

Mobile / Touch

Taps and gestures on the display itself

Most desktop systems provide both a shell and a GUI, precisely because neither suits every situation.

Why Power Users Still Reach for the CLI

- **Scriptability** — a sequence of shell commands can be saved as a script, automated, or chained with pipes; mouse clicks generally cannot
- **Precision** — a command plus its exact flags leaves no ambiguity; navigating menus can vary across versions and is easy to get subtly wrong
- **Remote accessibility** — a shell works perfectly over a slow or low-bandwidth remote connection (SSH into a server with no display), where a GUI would be impractical

The CLI's real advantage is repeatability, not speed

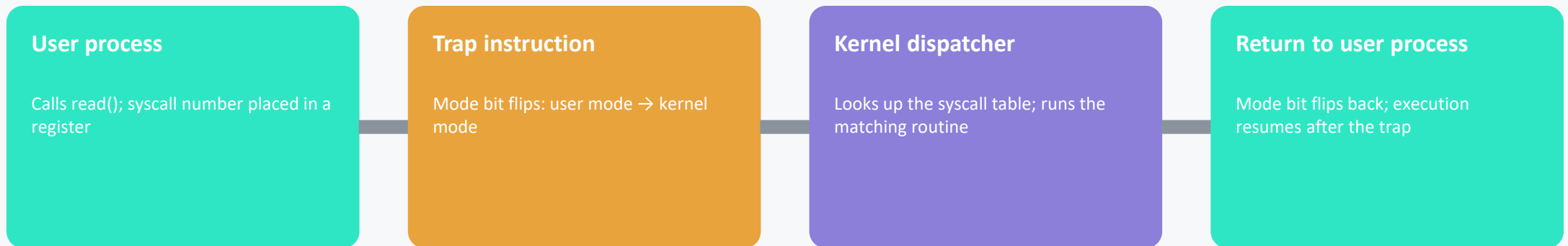
A single command typed once is not obviously faster than a single click. The CLI wins decisively the moment a task needs to happen more than once, or needs to happen identically every time — which is most of what system administration and development actually involve.

System Calls: The Only Legitimate Doorway

A system call is the programming interface through which a user program requests a service from the OS.

- **Not an ordinary function call** — it has to cross the protection boundary between user mode and kernel mode
- **Invoked through a wrapper** — most often from C or C++ via a small library wrapper function, though some low-level tasks still need assembly to issue the call directly
- **The literal “doorway”** — the only sanctioned way a user program can ask the OS to perform a privileged action on its behalf

How a System Call Transfers Control



A system call is a round trip, not a free function call

Every system call pays for two mode switches and the kernel actually doing the work. This is why reading a file one byte at a time via repeated `read()` calls is slow, while reading large buffered chunks is fast — fewer round trips through this sequence.

Passing Parameters to System Calls

A system call almost always needs parameters — which file to open, what to write, how many bytes to read. Three general methods pass them:

- **In registers** — the simplest method; limited by how many registers exist and how large a value each can hold
- **In a memory block, address passed in a register** — removes the limit on number or size of parameters entirely; the approach Linux and Solaris use
- **On the stack** — the calling program pushes parameters; the OS pops them off, at the cost of extra stack bookkeeping
- **Every method is still checked** — none is by itself a security guarantee; the kernel validates whatever it receives before acting on it

System Call Types and Examples

Category	Purpose	UNIX/Linux examples
Process control	Create/terminate a process, load and execute a program, get/set attributes, wait, allocate memory	fork(), exec(), exit(), wait()
File management	Create, delete, open, close, read, write, or reposition within a file; get/set attributes	open(), read(), write(), close()
Device management	Request or release a device, read/write to it, get/set attributes, attach/detach	ioctl()
Information maintenance	Get/set system time or date; get/set process, file, or device attributes	getpid(), alarm()
Communications	Create/destroy a connection, send/receive messages, transfer status information	pipe(), shmget(), mmap()
Protection	Control access to resources; get/set permissions on files or processes	chmod(), chown(), umask()

Worked Example: What a Shell Does When You Type “ls -l”

Running a single command at a shell prompt triggers at least two process-control system calls, working together:

```
pid_t pid = fork();           /* create a near-duplicate child process */

if (pid == 0) {
    /* child: replace this process's own memory image with /bin/ls */
    execlp("ls", "ls", "-l", NULL);
} else {
    /* parent (the shell): wait for the child to finish before showing a new prompt */
    wait(NULL);
}
```

`fork()` creates a new process that starts as a near-identical copy of the shell itself. `execlp()`, called only inside the child, discards that copy's memory image and replaces it with the `ls` program. The parent shell calls `wait()` so it pauses until the child finishes. Every command you've typed at a UNIX-family shell follows this `fork() + exec() + wait()` pattern.

Key Takeaways

- OS services split into those convenient for the user (execution, I/O, file-system manipulation, communications, error detection) and those that keep the system itself efficient (resource allocation, accounting, protection and security)
- CLI, GUI, and mobile/touch interfaces each suit different tasks; power users favor the CLI for scriptability, precision, and remote accessibility
- A system call crosses the user-mode/kernel-mode boundary through a trap: a syscall number identifies the request, a dispatcher looks it up in a syscall table, and control returns only once the kernel routine finishes
- Parameters reach a system call in registers, in a memory block whose address is passed in a register, or on the stack — the kernel validates them regardless
- System calls group into process control, file management, device management, information maintenance, communications, and protection — a single typed command already exercises several, via `fork()`, `exec()`, and `wait()`